Testes de Software

Objetivos da aula

- Entender o papel dos testes no desenvolvimento de software.
- Conhecer tipos e níveis de teste.
- Compreender o que são testes de unidade.
- Aprender a escrever e organizar testes com JUnit.
- Projetar bons casos de teste (equivalência e valores limite).
- Isolar dependências com dublês de teste e Mockito.
- Medir e interpretar a cobertura de código com JaCoCo.

Roteiro

1. Fundamentos: por que, tipos e níveis de teste
2. Teste de unidade, AAA, TDD e BDD
3. JUnit: configuração, anotações e verificações
4. Ciclo de vida, exceções e testes parametrizados
5. Projetando casos de teste
6. Isolamento: dublês e Mockito
7. Cobertura, maus cheiros e boas práticas
8. Atividade prática

Por que testar software?

- Para verificar se o sistema faz o que deveria fazer.
- Para encontrar erros antes que cheguem ao usuário.
- Para aumentar a confiança ao modificar o código.
- Para documentar o comportamento esperado do sistema.

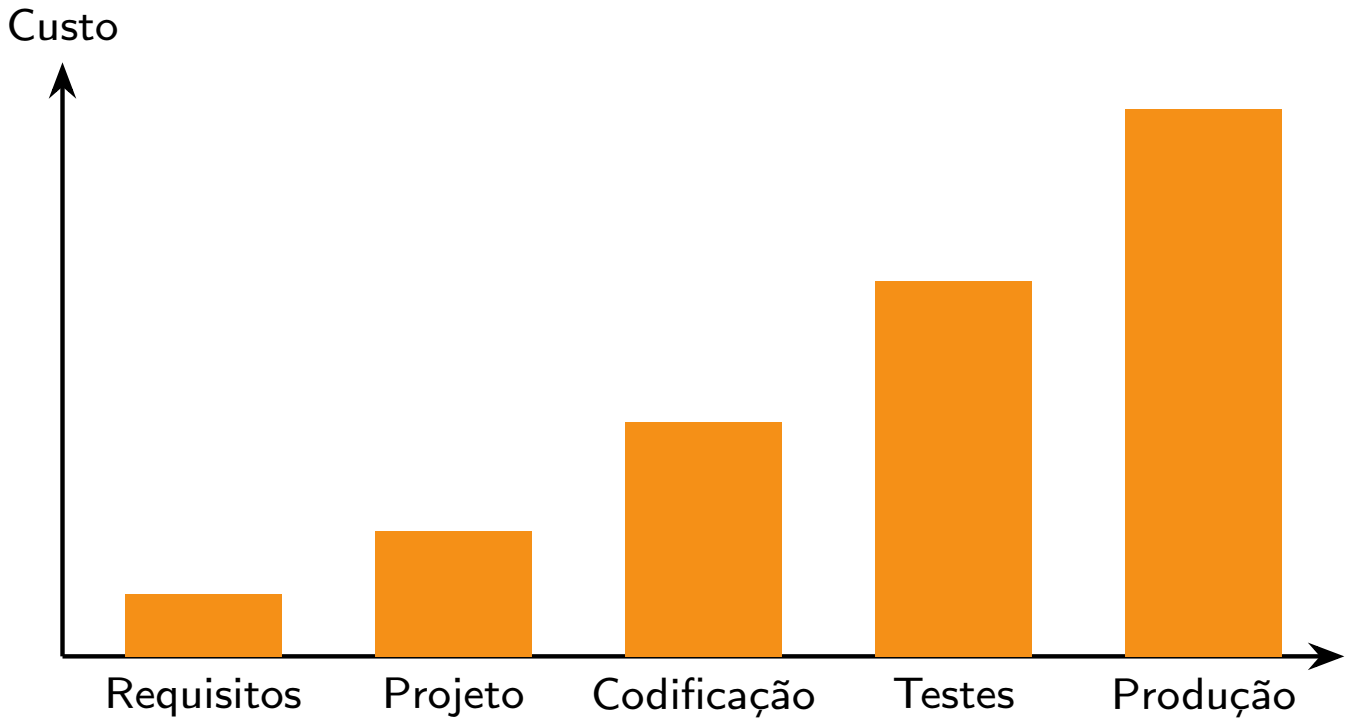
Teste não é apenas encontrar erro

- Testar também ajuda a entender o comportamento do sistema.
- Um bom teste descreve uma regra de negócio.
- Testes servem como documentação executável.
- Quanto mais cedo o erro aparece, mais barato é corrigir.

Limitação fundamental

Testes mostram a **presença** de defeitos, nunca a sua **ausência**. Passar em todos os testes não prova que o software está correto.

O custo de corrigir um erro



- O custo cresce muito à medida que o defeito é descoberto mais tarde.
- Ilustração qualitativa: as alturas são apenas ilustrativas.

Verificação e Validação

Verificação

"Estamos construindo o produto **corretamente**?"
O software está de acordo com a especificação?

Validação

"Estamos construindo o **produto certo**?"
O software atende à necessidade real do cliente?

- Testes de unidade: foco em verificação.
- Testes de aceitação: foco em validação.

Tipos de teste

- **Funcionais:** verificam funcionalidades do sistema.
- **Não funcionais:** avaliam desempenho, segurança, usabilidade etc.
- **Manuais:** realizados por pessoas.
- **Automatizados:** executados por ferramentas ou código.

Exemplos de testes não funcionais

- **Desempenho:** tempo de resposta com carga normal.
- **Carga e estresse:** comportamento com muitos usuários ou além do limite.
- **Segurança:** injeção de SQL, autenticação, autorização.
- **Usabilidade:** facilidade de uso da interface.
- **Compatibilidade:** navegadores, sistemas operacionais, versões.
- **Regressão:** garantir que o que funcionava continua funcionando.

Teste de regressão

Definição

Reexecutar testes existentes após uma mudança para garantir que nada que funcionava foi quebrado.

- É o principal benefício de ter testes automatizados.
- Cada correção de bug deve gerar um novo teste que reproduz o bug.
- Executados a cada commit em um servidor de integração contínua.

Técnicas: caixa-preta, branca e cinza

- **Caixa-preta:** testa entradas e saídas, sem olhar o código interno.
- **Caixa-branca:** usa o conhecimento da estrutura interna (desvios, laços).
- **Caixa-cinza:** combinação das duas abordagens.

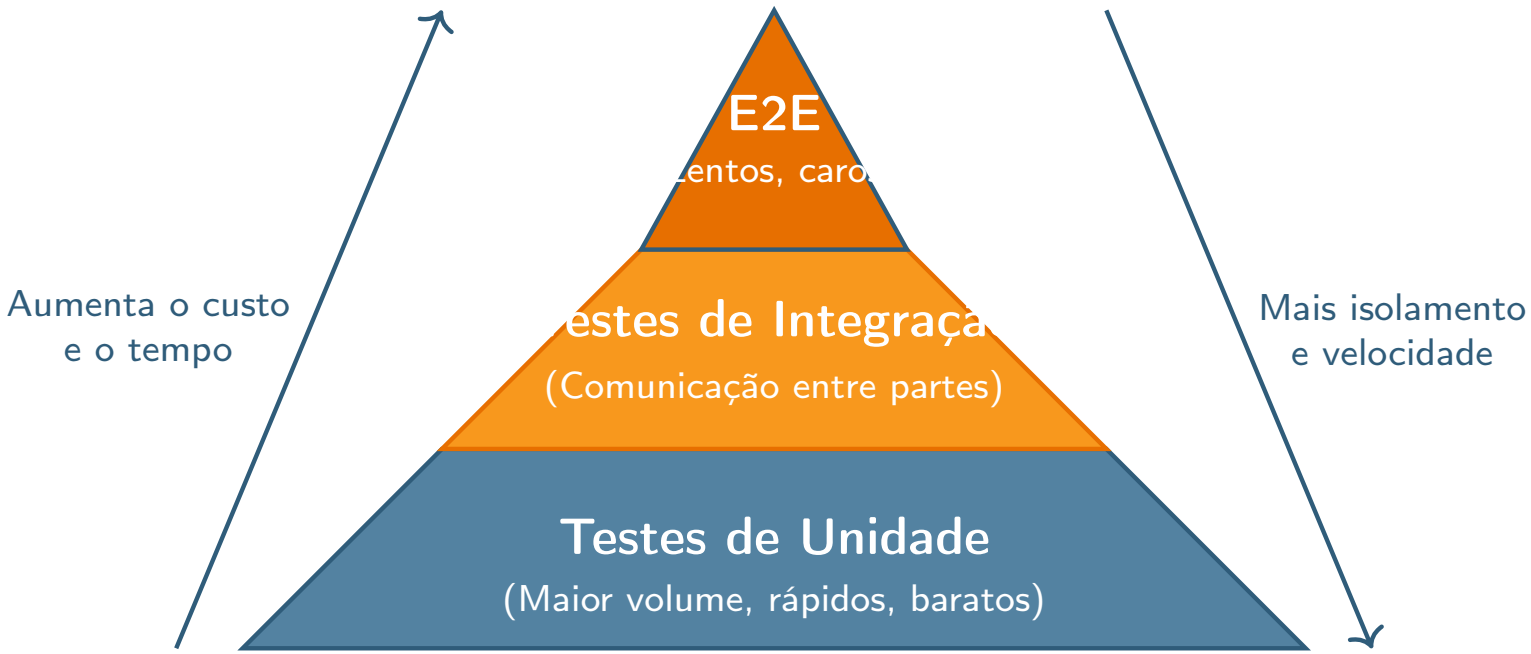
Dica

Testes de unidade costumam ser caixa-branca; testes de aceitação, caixa-preta.

Níveis de teste

- **Teste de unidade:** testa uma pequena parte isolada do código.
- **Teste de integração:** testa a comunicação entre partes.
- **Teste de sistema:** testa o sistema completo.
- **Teste de aceitação:** valida se atende às necessidades do cliente.

Pirâmide de Testes na Prática



Anti-padrão: o "cone de sorvete"

- Muitos testes manuais e de interface (E2E), poucos testes de unidade.
- Resultado: suíte lenta, frágil e cara de manter.
- Falhas difíceis de diagnosticar: o erro pode estar em qualquer camada.

Regra prática

Prefira muitos testes pequenos e rápidos na base e poucos testes amplos no topo.

Foco da aula

Nesta aula

O foco será em **testes de unidade**, usando Java e JUnit.

- Como criar um teste.
- Como organizar o código de teste.
- Como usar anotações.
- Como verificar resultados esperados.
- Como isolar dependências e medir cobertura.

O que é teste de unidade?

- É um teste feito sobre uma unidade pequena do sistema.
- Normalmente testa um método ou uma classe.
- Deve ser rápido.
- Deve depender o mínimo possível de banco de dados, rede ou arquivos.

Exemplo de unidade

- Uma classe `Calculadora`.
- Um método `somar`.
- Um método `dividir`.
- Uma regra como: “não permitir divisão por zero”.

Características de um bom teste: F.I.R.S.T.

- **F**ast (rápido): executa em milissegundos.
- **I**ndependent (independente): não depende de outros testes.
- **R**epeatable (repetível): mesmo resultado em qualquer ambiente.
- **S**elf-validating (autoverificável): passa ou falha sozinho, sem inspeção humana.
- **T**imely (oportuno): escrito junto com o código (ou antes dele).

Classe que será testada

```java
public class Calculadora {

    public int somar(int a, int b) {
        return a + b;
    }

    public int subtrair(int a, int b) {
        return a - b;
    }

    public int dividir(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Divisor invalido");
        }

        return a / b;
    }
}
```

O que um teste precisa ter?

- Um cenário inicial.
- Uma ação executada.
- Um resultado esperado.
- Uma verificação automática.

Padrão AAA

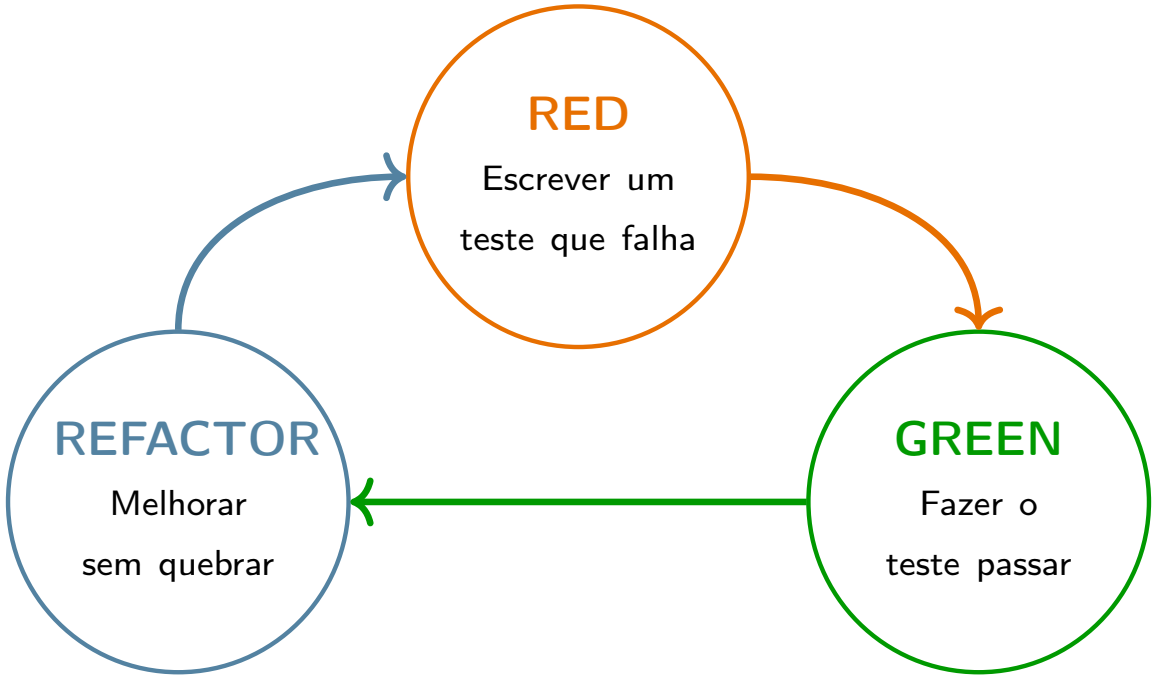
- **Arrange:** preparar os dados do teste.
- **Act:** executar a ação testada.
- **Assert:** verificar se o resultado está correto.

Ideia

Um teste bom é fácil de ler e mostra claramente o comportamento esperado.

Test-Driven Development (TDD)

- Inverte a ordem clássica: escrevemos o teste **antes** do código de produção.
- Guia o design do software e gera alta cobertura de testes.



TDD: as três regras e um exemplo

1. Não escreva código de produção sem antes ter um teste que falha.
2. Escreva apenas o teste suficiente para falhar (não compilar já é falhar).
3. Escreva apenas o código suficiente para passar no teste.

Exemplo com a Calculadora

1) Escrevo `deveSomarDoisNumeros` (falha: o método não existe).
2) Crio `somar` retornando `a + b` (verde).
3) Refatoro, se necessário, e repito com `dividir`.

Behavior-Driven Development (BDD)

- Evolução do TDD: o foco passa de "testar a implementação" para **testar o comportamento**.
- Aproxima desenvolvedores, testadores e especialistas do negócio usando uma linguagem comum.

Paralelo com o Padrão AAA

- **Given (Dado):** contexto inicial $\rightarrow$ *Arrange*
- **When (Quando):** a ação ocorre $\rightarrow$ *Act*
- **Then (Então):** resultado esperado $\rightarrow$ *Assert*

O que é o JUnit?

- Framework de testes automatizados para Java.
- Descobre e executa métodos de teste e relata o resultado.
- Integrado às IDEs, ao Maven, ao Gradle e a servidores de CI.

Versão usada nesta aula: JUnit 4

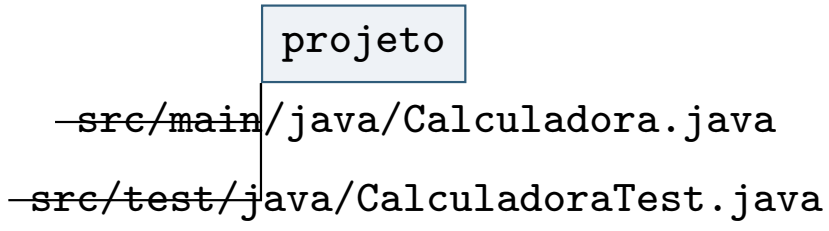
Todos os exemplos usam **JUnit 4.13** (pacote `org.junit`). O JUnit 5 usa outro pacote e outras anotações (veja o slide de comparação).

JUnit 4 × JUnit 5

Conceito	**JUnit 4**	**JUnit 5**
Pacote	`org.junit`	`org.junit.jupiter.api`
Antes de cada teste	`@Before`	`@BeforeEach`
Depois de cada teste	`@After`	`@AfterEach`
Uma vez, antes de todos	`@BeforeClass`	`@BeforeAll`
Uma vez, depois de todos	`@AfterClass`	`@AfterAll`
Ignorar	`@Ignore`	`@Disabled`
Visibilidade do método	`public`	pode ser pacote
Exceções	`assertThrows` (4.13)	`assertThrows`

Onde ficam os testes?

- `src/main/java`: código de produção.
- `src/test/java`: código de teste.
- Mesmo pacote da classe testada.
- Classe `Calculadora` → `CalculadoraTest`.



Dica

O Maven executa por padrão classes terminadas em `Test`.

Primeiro teste com JUnit

```java
import org.junit.Test;

import static org.junit.Assert.assertEquals;

public class CalculadoraTest {

    @Test
    public void deveSomarDoisNumeros() {
        Calculadora calculadora = new Calculadora();

        int resultado = calculadora.somar(2, 3);

        assertEquals(5, resultado);
    }
}
```

O que é `@Test`?

- Indica que o método é um teste.
- O JUnit executa automaticamente métodos marcados com `@Test`.
- O teste passa se nenhuma verificação falhar e nenhuma exceção escapar.

Regras do JUnit 4 para um método de teste

Deve ser `public`, `void` e **sem parâmetros**. Caso contrário, o JUnit lança `InitializationError` ("Method ... should be public").

Resultado de um teste: sucesso, falha e erro

- **Sucesso (verde):** todas as verificações passaram.
- **Falha (failure):** uma verificação (`assert`) não foi satisfeita → `AssertionError`.
- **Erro (error):** exceção inesperada durante o teste, como `NullPointerException`.
- **Ignorado (skipped):** teste marcado com `@Ignore`.

Dica

Falha indica comportamento diferente do esperado; erro indica problema no código ou no próprio teste.

Principais verificações

- `assertEquals`: verifica se dois valores são iguais.
- `assertNotEquals`: verifica se dois valores são diferentes.
- `assertTrue` / `assertFalse`: condição verdadeira / falsa.
- `assertNull` / `assertNotNull`: objeto nulo / não nulo.
- `assertSame` / `assertNotSame`: mesma referência / referências diferentes.
- `assertArrayEquals`: arrays com o mesmo conteúdo.
- `assertThrows`: uma exceção foi lançada.
- `fail`: falha o teste explicitamente.

Usando assertEquals

```java
@Test
public void deveSubtrairDoisNumeros() {
    Calculadora calculadora = new Calculadora();

    int resultado = calculadora.subtrair(10, 4);

    assertEquals(6, resultado);
}
```

Ordem dos argumentos

Sempre `assertEquals(esperado, atual)`. Invertendo, a mensagem de falha fica enganosa.

Comparando números decimais

```java
@Test
public void deveSomarDecimais() {
    double resultado = 0.1 + 0.2;

    // errado: 0.1 + 0.2 nao e exatamente 0.3
    // assertEquals(0.3, resultado);

    assertEquals(0.3, resultado, 0.0001); // com tolerancia (delta)
}
```

- `float` e `double` têm erro de representação binária.
- Use sempre o terceiro argumento (*delta*) ou `BigDecimal` para valores monetários.

assertTrue e assertFalse

```
@Test
public void deveIdentificarNumeroPar() {
    assertTrue(NumeroUtil.ehPar(4));
    assertFalse(NumeroUtil.ehPar(7));
}
```

- Use para resultados `boolean`.
- Prefira `assertEquals` quando houver um valor concreto a comparar: a mensagem de falha é mais informativa.

assertNull e assertNotNull

```java
@Test
public void deveRetornarNuloQuandoClienteNaoExiste() {
    Cliente c = repositorio.buscarPorId(999);

    assertNull(c);
}

@Test
public void deveRetornarClienteQuandoExiste() {
    Cliente c = repositorio.buscarPorId(1);

    assertNotNull(c);
    assertEquals("Maria", c.getNome());
}
```

assertEquals × assertSame

```
String a = new String("java");
String b = new String("java");

assertEquals(a, b);       // passa: usa equals()
assertNotSame(a, b);      // passa: objetos diferentes (==)
assertSame(a, a);         // passa: mesma referencia
```

- assertEquals usa equals(): compara o **conteúdo**.
- assertSame usa ==: compara a **referência**.
- Para seus objetos, sobrescreva equals e hashCode.

Comparando arrays e coleções

```java
@Test
public void deveOrdenarArray() {
    int[] entrada = {3, 1, 2};

    Ordenador.ordenar(entrada);

    assertArrayEquals(new int[] {1, 2, 3}, entrada);
}

@Test
public void deveListarNomes() {
    List<String> nomes = servico.listarNomes();

    assertEquals(Arrays.asList("Ana", "Bia"), nomes);
}
```

- `assertEquals` em arrays compara referências: use `assertArrayEquals`.

Mensagens de falha personalizadas

```
assertEquals("Soma incorreta para 2 + 3", 5, resultado);

assertTrue("Pedido de valor positivo deveria ser aprovado",
           service.pedidoAprovado(100.0));
```

- No JUnit 4 a mensagem é o **primeiro** argumento (opcional).
- Ajuda a diagnosticar a falha sem abrir o código do teste.
- Não abuse: um nome de teste bom já explica o cenário.

Verificações com Hamcrest (assertThat)

```
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.*;

assertThat(resultado, is(5));
assertThat(nome, startsWith("Ma"));
assertThat(lista, hasSize(3));
assertThat(lista, hasItem("Ana"));
assertThat(valor, greaterThan(0.0));
```

- Leitura mais próxima da linguagem natural.
- Requer a dependência `hamcrest` (`org.hamcrest.Matchers`).

Testando exceções com assertThrows

```java
import static org.junit.Assert.assertThrows; // JUnit 4.13+

@Test
public void naoDeveDividirPorZero() {
    Calculadora calculadora = new Calculadora();

    assertThrows(
        IllegalArgumentException.class,
        () -> calculadora.dividir(10, 0)
    );
}
```

- Passa somente se o tipo de exceção esperado for lançado.
- Requer JUnit **4.13** ou superior e Java 8+ (lambda).

Verificando a mensagem da exceção

```java
@Test
public void deveInformarMensagemAoDividirPorZero() {
    Calculadora calculadora = new Calculadora();

    IllegalArgumentException e = assertThrows(
        IllegalArgumentException.class,
        () -> calculadora.dividir(10, 0)
    );

    assertEquals("Divisor invalido", e.getMessage());
}
```

- `assertThrows` **retorna** a exceção capturada.
- Assim é possível verificar mensagem, causa ou campos específicos.

Outras formas de testar exceções (JUnit 4)

```java
// 1) atributo expected: simples, mas pouco preciso
@Test(expected = IllegalArgumentException.class)
public void naoDeveDividirPorZero() {
    new Calculadora().dividir(10, 0);
}

// 2) try/catch com fail(): funciona em qualquer versao
@Test
public void naoDeveDividirPorZeroComTryCatch() {
    try {
        new Calculadora().dividir(10, 0);
        fail("Deveria lancar IllegalArgumentException");
    } catch (IllegalArgumentException e) {
        assertEquals("Divisor invalido", e.getMessage());
    }
}
```

Também testamos que **não** há exceção

```java
@Test
public void deveDividirSemLancarExcecao() {
    Calculadora calculadora = new Calculadora();

    int resultado = calculadora.dividir(10, 2);

    assertEquals(5, resultado);
}
```

- Se uma exceção inesperada escapar, o JUnit marca o teste como **erro**.
- Não é preciso envolver o código em `try/catch` só para "garantir" que não lança.

O que é preparação do teste?

- Muitos testes precisam criar os mesmos objetos.
- Repetir essa criação em todos os métodos deixa o código poluído.
- Para isso, usamos métodos de preparação.
- No JUnit 4, usamos `@Before` para preparar cada teste.

Cuidado: não existe `@Setup`

Erro comum

Não existe anotação `@Setup` no JUnit. Em alguns frameworks há um método chamado `setUp()`, mas ele só funciona por convenção de nome (JUnit 3).

- JUnit 4: `@Before` (antes de cada teste).
- JUnit 5: `@BeforeEach`.
- Para preparar dados uma única vez: `@BeforeClass` (JUnit 4).

Usando @Before

```java
import org.junit.Before;
import org.junit.Test;

import static org.junit.Assert.assertEquals;

public class CalculadoraTest {

    private Calculadora calculadora;

    @Before
    public void preparar() {
        calculadora = new Calculadora();
    }

    @Test
    public void deveSomarDoisNumeros() {
        int resultado = calculadora.somar(2, 3);

        assertEquals(5, resultado);
    }
}
```

O que é `@Before`?

- Executa antes de cada método marcado com `@Test`.
- Garante um estado limpo para cada teste.
- Evita repetição de código.
- Ajuda a manter os testes independentes.
- O método deve ser `public void` e sem parâmetros.

O que é `@After`?

- Executa depois de cada teste, mesmo que ele falhe.
- Pode ser usado para limpar recursos.
- Exemplo: fechar arquivos, limpar variáveis ou remover dados temporários.
- Em testes simples, muitas vezes não é necessário.

Exemplo com @After

```java
import org.junit.After;

public class ArquivoServiceTest {

    private File temporario;

    @Before
    public void preparar() throws IOException {
        temporario = File.createTempFile("teste", ".txt");
    }

    @After
    public void finalizar() {
        temporario.delete();    // limpa o recurso
    }
}
```

@BeforeClass e @AfterClass

```java
public class ConexaoTest {

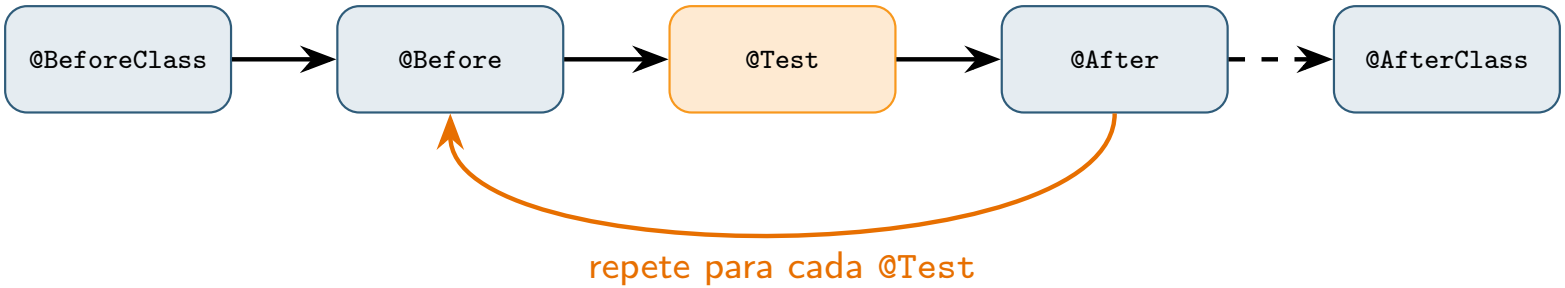
    private static Conexao conexao;

    @BeforeClass
    public static void abrirConexao() {    // uma vez, antes de
        todos
        conexao = Conexao.abrir();
    }

    @AfterClass
    public static void fecharConexao() {  // uma vez, depois de
        todos
        conexao.fechar();
    }
}
```

- Os métodos devem ser `public static void`.
- Use para recursos **caros** que podem ser compartilhados com segurança.

Ciclo de vida de uma classe de teste



- `@BeforeClass` e `@AfterClass`: uma vez por classe.
- `@Before`, `@Test`, `@After`: uma vez por teste.

Uma nova instância por teste

- O JUnit cria **um novo objeto** da classe de teste para **cada** método `@Test`.
- Atributos de instância não "vazam" de um teste para outro.
- Atributos `static` são compartilhados: cuidado.

Armadilha

Um atributo `static` alterado por um teste pode influenciar outro e criar dependência de ordem de execução.

Ignorando e limitando o tempo de um teste

```java
@Ignore("Aguardando correcao do bug #123")
@Test
public void testeTemporariamenteDesativado() { ... }

@Test(timeout = 500)    // milissegundos
public void deveTerminarRapido() {
    assertEquals(5050, Soma.ate(100));
}
```

- `@Ignore`: sempre informe o motivo; revise periodicamente.
- `timeout`: falha se o teste demorar mais que o limite (útil contra laços infinitos).

Independência dos testes

- Um teste não deve depender da execução de outro.
- A ordem dos testes não deve importar.
- Cada teste deve preparar seus próprios dados.
- Isso evita resultados imprevisíveis.

E a ordem de execução?

- No JUnit 4 a ordem é determinística, porém **não previsível**.
- Existe `@FixMethodOrder`, mas serve para casos muito específicos.
- Se um teste só passa depois de outro, o problema está no design dos testes.

Dica

Teste de verdade: execute um único teste isolado. Se ele falhar sozinho, ele depende de algo que não deveria.

Nomeando testes

- O nome do teste deve explicar o comportamento esperado.
- Evite nomes genéricos como `teste1`.
- Padrões comuns:
  - `deveSomarDoisNumeros`
  - `naoDeveDividirPorZero`
  - `deveRetornarDescontoQuandoClienteForVip`
  - `dividir_divisorZero_lancaExcecao`

O que testar?

- Casos comuns (caminho feliz).
- Casos extremos (limites).
- Entradas inválidas (caminhos tristes).
- Regras de negócio importantes.
- Situações que já causaram erro antes.
- Valores nulos, vazios, negativos e zero.

Partição de equivalência

Definição

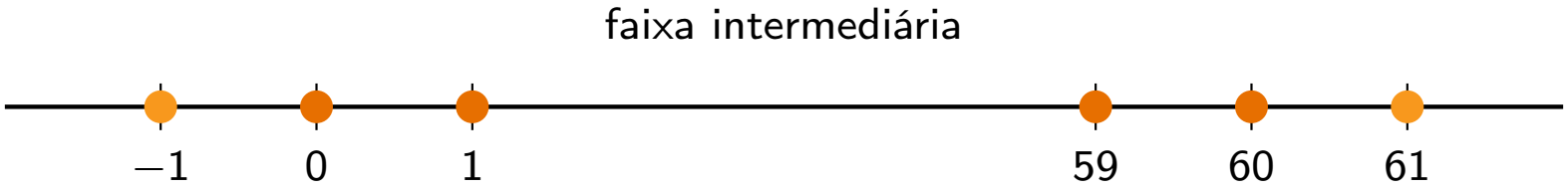
Dividir o domínio de entrada em grupos (classes) em que o programa deve se comportar da mesma forma. Basta testar **um representante** de cada classe.

Exemplo: desconto por idade

Idade < 0: inválida | 0 a 17: sem desconto | 18 a 59: normal | ≥ 60: idoso.
Testar 1 valor de cada faixa: -5, 10, 30, 70.

Análise de valores limite

- A maioria dos defeitos aparece **nas fronteiras** entre as classes.
- Teste o valor da fronteira, um acima e um abaixo.



Valores limite na prática

```
// Regra: aprovado somente se valor > 0
@Test public void naoAprovaValorNegativo()    { assertFalse(s.
    pedidoAprovado(-0.01)); }
@Test public void naoAprovaValorZero()        { assertFalse(s.
    pedidoAprovado(0.0)); }
@Test public void aprovaMenorValorPositivo() { assertTrue (s.
    pedidoAprovado(0.01)); }
@Test public void aprovaValorAlto()           { assertTrue (s.
    pedidoAprovado(1_000_000)); }
```

- O erro clássico > no lugar de >= só aparece no teste de fronteira.

Caminho feliz e caminhos tristes

Caminho feliz

Entradas válidas e fluxo normal.
`dividir(10, 2)` $\rightarrow 5$

Caminhos tristes

Entradas inválidas, nulos, limites e exceções.
`dividir(10, 0)` $\rightarrow$ exceção

- Equipes iniciantes testam quase só o caminho feliz.
- É nos caminhos tristes que os defeitos costumam se esconder.

Tabela de casos para `Calculadora`

Método	**Entrada**	**Esperado**	**Categoria**
`somar`	2, 3	5	comum
`somar`	0, 0	0	neutro
`somar`	-1, 1	0	negativo
`somar`	`MAX_VALUE`, 1	*overflow*	limite
`subtrair`	10, 4	6	comum
`dividir`	10, 2	5	comum
`dividir`	7, 2	3	divisão inteira
`dividir`	10, 0	exceção	inválida

Classe de teste completa

```
public class CalculadoraTest {

    private Calculadora calculadora;

    @Before
    public void preparar() { calculadora = new Calculadora(); }

    @Test public void deveSomarDoisPositivos()  { assertEquals(5, calculadora.somar(2, 3)
        ); }
    @Test public void deveSomarZeros()          { assertEquals(0, calculadora.somar(0, 0)
        ); }
    @Test public void deveSomarComNegativo()    { assertEquals(0, calculadora.somar(-1,
        1)); }
    @Test public void deveSubtrair()            { assertEquals(6, calculadora.subtrair
        (10, 4)); }
    @Test public void deveDividirExato()        { assertEquals(5, calculadora.dividir(10,
         2)); }
    @Test public void deveTruncarDivisaoInteira() { assertEquals(3, calculadora.dividir
        (7, 2)); }

    @Test
    public void naoDeveDividirPorZero() {
        assertThrows(IllegalArgumentException.class, () -> calculadora.dividir(10, 0));
    }
}
```

Testes parametrizados: o problema

- Vários testes quase idênticos, mudando só os valores.
- Muita repetição e pouca clareza.
- Solução: um único teste executado com **várias linhas de dados**.

```
1 @Test public void somaA() { assertEquals(3, calculadora.somar(1, 2)
      ); }
2 @Test public void somaB() { assertEquals(0, calculadora.somar(0, 0)
      ); }
3 @Test public void somaC() { assertEquals(0, calculadora.somar(-1,
      1)); }
```

Testes parametrizados com JUnit 4

```java
@RunWith(Parameterized.class)
public class SomarParametrizadoTest {

    @Parameters(name = "{0} + {1} = {2}")
    public static Collection<Object[]> dados() {
        return Arrays.asList(new Object[][] {
            {1, 2, 3}, {0, 0, 0}, {-1, 1, 0}, {100, 200, 300}
        });
    }

    private final int a, b, esperado;

    public SomarParametrizadoTest(int a, int b, int esperado) {
        this.a = a; this.b = b; this.esperado = esperado;
    }

    @Test
    public void deveSomar() {
        assertEquals(esperado, new Calculadora().somar(a, b));
    }
}
```

Agrupando testes em suítes

```
@RunWith(Suite.class)
@Suite.SuiteClasses({
    CalculadoraTest.class,
    PedidoServiceTest.class,
    DescontoServiceTest.class
})
public class TodosOsTestes { }
```

- Executa várias classes de teste de uma vez.
- Com Maven ou Gradle, normalmente não é necessário: o plugin descobre os testes sozinho.

O problema das dependências

- Classes reais dependem de banco de dados, rede, arquivos, relógio, e-mail...
- Isso torna o teste **lento**, **instável** e difícil de reproduzir.
- Um teste de unidade deve falhar **apenas** por defeito na unidade testada.

Analogia

Testar o motor de um carro: não é preciso ligar o carro na estrada. Basta uma bancada que simula as peças ao redor.

Dublês de teste (*test doubles*)

- **Dummy:** objeto passado só para preencher um parâmetro; nunca é usado.
- **Stub:** devolve respostas prontas e fixas.
- **Fake:** implementação simplificada que funciona (ex.: repositório em memória).
- **Spy:** objeto real que registra como foi usado.
- **Mock:** objeto programado com expectativas; o teste verifica as interações.

Exemplo: serviço com dependência

```java
public interface Notificador {
    void enviar(String destino, String mensagem);
}

public class CheckoutService {

    private final Notificador notificador;

    public CheckoutService(Notificador notificador) {
        this.notificador = notificador;
    }

    public boolean finalizar(double valor, String email) {
        if (valor <= 0) {
            return false;
        }
        notificador.enviar(email, "Pedido aprovado");
        return true;
    }
}
```

Testando com Mockito

```java
import static org.mockito.Mockito.*;

public class CheckoutServiceTest {

    private Notificador notificador;
    private CheckoutService service;

    @Before
    public void preparar() {
        notificador = mock(Notificador.class);        // duble criado pelo Mockito
        service = new CheckoutService(notificador);
    }

    @Test
    public void deveNotificarQuandoPedidoAprovado() {
        boolean ok = service.finalizar(100.0, "a@b.com");

        assertTrue(ok);
        verify(notificador).enviar("a@b.com", "Pedido aprovado");
    }

    @Test
    public void naoDeveNotificarQuandoValorInvalido() {
        service.finalizar(0.0, "a@b.com");

        verify(notificador, never()).enviar(anyString(), anyString());
    }
}
```

Stubs com `when(...).thenReturn(...)`

```
RepositorioCliente repo = mock(RepositorioCliente.class);

when(repo.buscarPorId(1)).thenReturn(new Cliente("Maria", true));
when(repo.buscarPorId(99)).thenReturn(null);
when(repo.buscarPorId(-1)).thenThrow(new IllegalArgumentException()
    );

// o servico sob teste usa o repo falso, sem banco de dados
```

- Define o que o dublê responde para cada entrada.
- Permite simular situações difíceis de reproduzir: erro de rede, banco indisponível.

Estado ou interação? Quando usar mocks

Verificar estado

Chama o método e confere o **resultado** ou o estado do objeto.
Preferível sempre que possível.

Verificar interação

Confere se uma **chamada** ocorreu (`verify`).
Útil quando o efeito é externo (e-mail, log).

Cuidado

Mocks em excesso acoplam o teste à implementação: ao refatorar, o teste quebra mesmo com o comportamento correto.

Fontes de não determinismo (testes instáveis)

- **Data e hora atuais** (`LocalDate.now()`): injete um `Clock`.
- **Números aleatórios**: injete a fonte ou use uma semente fixa.
- **Threads e** `Thread.sleep`: gera testes lentos e intermitentes.
- **Rede, banco, sistema de arquivos**: use dublês ou ambientes controlados.
- **Ordem de coleções** (`HashSet`, `HashMap`): não assuma ordem.

Dica

Teste que ora passa, ora falha, sem mudança no código, é chamado *flaky*. Corrija ou remova: ele destrói a confiança na suíte.

Exemplo de regra de negócio

Regra

Um pedido só pode ser considerado aprovado se o valor for maior que zero.

- Valor positivo: pedido aprovado.
- Valor zero: pedido não aprovado.
- Valor negativo: pedido não aprovado.

Classe PedidoService

```
1 public class PedidoService {
2
3     public boolean pedidoAprovado(double valor) {
4         return valor > 0;
5     }
6 }
```

Teste da regra de negócio

```java
import org.junit.Test;

import static org.junit.Assert.*;

public class PedidoServiceTest {

    @Test
    public void deveAprovarPedidoComValorPositivo() {
        PedidoService service = new PedidoService();

        boolean resultado = service.pedidoAprovado(100.0);

        assertTrue(resultado);
    }

    @Test
    public void naoDeveAprovarPedidoComValorZero() {
        PedidoService service = new PedidoService();

        boolean resultado = service.pedidoAprovado(0.0);

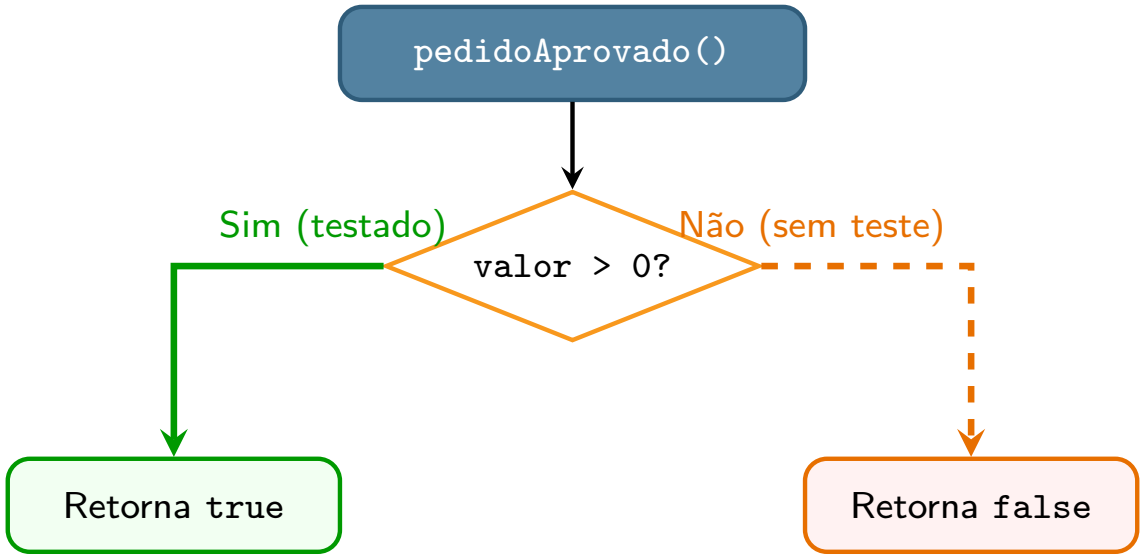
        assertFalse(resultado);
    }

    @Test
    public void naoDeveAprovarPedidoComValorNegativo() {
        assertFalse(new PedidoService().pedidoAprovado(-10.0));
    }
}
```

O que é Cobertura de Código?

- Métrica que avalia quanto do código-fonte é executado durante os testes.
- Ajuda a identificar áreas do sistema que não estão sendo validadas.
- **Principais métricas:**
  - **Linhas:** quantas linhas foram executadas?
  - **Métodos:** quantos métodos foram chamados?
  - **Ramificações (*branches*):** todos os caminhos de decisão (ex.: `if/else`) foram percorridos?

Entendendo a Cobertura de Ramificações



No ecossistema Java

A ferramenta mais usada é o **JaCoCo** (Java Code Coverage). Integra-se ao Maven ou Gradle e gera relatórios (HTML) com as ramificações.

Configurando o JaCoCo no Maven

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution>
      <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
      <id>report</id>
      <phase>test</phase>
      <goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

- Execute `mvn test`; o relatório fica em `target/site/jacoco/index.html`.

Exemplo Prático: Relatório JaCoCo na IDE

Insira aqui a captura de tela do relatório
(arquivo `jacoco-report.png`)

Figura: Visão de cobertura utilizando plugin na IDE

Entendendo o Relatório

- **Navegação hierárquica:** do macro ao micro (Pacote → Classe → Método).
- **Cores:**
  - **Verde:** instruções executadas pelos testes.
  - **Amarelo:** desvio parcialmente coberto (só um lado do `if`).
  - **Vermelho:** nunca executadas.
- **Análise da imagem:**
  - `User.java`: **100%** (*getters* e *setters* testados).
  - `HibernateUtil.java`: **80%**; `openSession()` e o construtor estão em vermelho.
- **Instructions:** contagem por instruções de *bytecode*, mais precisa que linhas.

O Mito dos 100% de Cobertura

Atenção: falsa sensação de segurança

100% de cobertura significa apenas que o código foi **executado**, não que o comportamento está **correto**.

- Um teste que chama o método sem nenhum `assert` gera cobertura, mas é inútil.
- **Onde focar:** regras de negócio complexas, caminhos tristes e casos de borda.
- Use a cobertura para achar **código que precisa de atenção**, não como troféu.

Exemplo: cobertura alta, teste inútil

```java
@Test
public void testeInutil() {
    PedidoService service = new PedidoService();

    service.pedidoAprovado(100.0);    // executa o codigo...
                                      // ...mas nao verifica nada!
}
```

Atenção

Esse teste nunca falha, mesmo que a regra seja alterada para `valor < 0`. Cobertura mede **execução**; quem verifica correção é o `assert`.

Teste de mutação (visão geral)

- Ferramentas (ex.: PIT) **alteram o código** de propósito (> vira >=, + vira -).
- Se nenhum teste falhar, o "mutante" **sobreviveu**: faltou verificação.
- Mede a **qualidade** dos testes, complementando a cobertura.

Analogia

É como testar o detector de fumaça acendendo um fósforo: se não apitar, o detector não presta.

Maus cheiros em testes (*test smells*)

- **Sem verificação:** executa o código mas não faz `assert`.
- **Assertion roulette:** muitos `assert` sem mensagem; difícil saber qual falhou.
- **Lógica condicional:** `if`, `for` dentro do teste.
- **Dados compartilhados:** testes que alteram o mesmo estado.
- **Teste frágil:** quebra a cada refatoração, sem mudança de comportamento.
- **Teste lento:** usa `sleep`, banco real ou rede.
- `@Ignore` **eterno:** testes desativados e esquecidos.

Boas práticas

- Teste uma coisa por vez.
- Use nomes claros.
- Mantenha os testes pequenos.
- Evite lógica complexa dentro do teste.
- Teste comportamento, não implementação interna.
- Trate o código de teste com o mesmo cuidado do código de produção.
- Corrija imediatamente testes que falham; nunca conviva com "vermelho".

Como executar os testes

- **Na IDE:** botão direito na classe ou método → *Run as JUnit Test*.
- **Maven:** `mvn test` (um teste: `mvn -Dtest=CalculadoraTest test`).
- **Gradle:** `gradle test`.
- **Integração contínua:** a cada *push*, o servidor (GitHub Actions, Jenkins etc.) executa toda a suíte.

Dica

Quanto mais rápido for rodar os testes, mais vezes eles serão executados.

Erros comuns de quem está começando

- Método de teste sem `public` (JUnit 4).
- Importar `org.junit.Test` junto com anotações do JUnit 5 (`@BeforeEach`).
- Trocar a ordem `esperado` / `atual`.
- Comparar `double` sem *delta*.
- Usar `assertEquals` para comparar arrays.
- Esquecer `@Test` (o método nunca roda).
- Colocar o teste em `src/main` em vez de `src/test`.

Checklist antes de entregar o teste

Checklist

- ☐ O nome descreve o comportamento esperado?
- ☐ Segue o padrão Arrange–Act–Assert?
- ☐ Existe pelo menos uma verificação significativa?
- ☐ Cobre caminho feliz, limites e entradas inválidas?
- ☐ É independente e repetível?
- ☐ Roda em milissegundos, sem banco ou rede?

Resumo

- Testes ajudam a aumentar a qualidade do software.
- Existem diferentes tipos e níveis de teste; a base da pirâmide são os testes de unidade.
- O JUnit permite automatizar testes em Java.
- `@Test`, `@Before`, `@After`, `@BeforeClass` e `@AfterClass` controlam o ciclo de vida.
- Bons casos de teste usam equivalência, valores limite e caminhos tristes.
- Dublês (Mockito) isolam dependências externas.
- Cobertura mostra o que não foi testado, não se o teste é bom.

Atividade prática

Proposta

Criar testes de unidade para uma classe responsável por calcular desconto.

- Cliente comum: sem desconto.
- Cliente VIP: 10% de desconto.
- Valor negativo: deve lançar exceção.
- Usar `@Test`, `@Before` e `assertThrows`.
- **Desafio:** valor zero, valores muito grandes e cliente nulo.
- **Desafio 2:** converter em teste parametrizado.
- **Desafio 3:** medir a cobertura com JaCoCo e alcançar 100% de ramificações.

Atividade: classe sugerida

```java
public class CalculadoraDesconto {

    public double calcular(double valor, boolean vip) {
        if (valor < 0) {
            throw new IllegalArgumentException("Valor invalido");
        }
        if (vip) {
            return valor * 0.10;    // desconto de 10%
        }
        return 0.0;
    }
}
```

- *Pergunta:* que casos precisam existir para cobrir **todos** os ramos?

Atividade: possível solução

```
public class CalculadoraDescontoTest {

    private CalculadoraDesconto calc;

    @Before
    public void preparar() { calc = new CalculadoraDesconto(); }

    @Test public void clienteComumNaoTemDesconto() { assertEquals(0.0,  calc.calcular
          (100.0, false), 0.001); }
    @Test public void clienteVipTemDezPorCento()   { assertEquals(10.0, calc.calcular
          (100.0, true),  0.001); }
    @Test public void valorZeroNaoGeraDesconto()   { assertEquals(0.0,  calc.calcular
          (0.0, true),    0.001); }
    @Test public void valorGrandeProporcional()    { assertEquals(1.0E6, calc.calcular
          (1.0E7, true), 0.001); }

    @Test
    public void valorNegativoDeveLancarExcecao() {
        assertThrows(IllegalArgumentException.class, () -> calc.calcular(-1.0, true));
    }
}
```

Para pensar

- Por que 100% de cobertura não garante um software correto?
- Qual a diferença entre *falha* e *erro* em um teste JUnit?
- Quando um mock ajuda e quando ele atrapalha?
- Por que um teste não deve depender da ordem de execução?
- Que valores você testaria para uma regra do tipo `idade >= 18`?

Referências

- JUnit 4: `junit.org/junit4`
- JUnit 5: `junit.org/junit5`
- Mockito: `site.mockito.org`
- JaCoCo: `jacoco.org`
- Beck, K. *Test-Driven Development: By Example*. Addison-Wesley.
- Meszaros, G. *xUnit Test Patterns*. Addison-Wesley.
- Martin, R. C. *Clean Code* (cap. 9: Testes de Unidade). Alta Books.